Robot CLI Command Tool

Robot CLI Command Tool

1. Course Content
    - Course Overview
    - Learning Objectives
2. Preparation
3. Introduction to CLI Tools
    - 3.1 What is a Robot CLI Control Tool?
    - 3.2 Why are CLI command tools needed?
4. Communication Architecture for CLI-Controlled Robots
5. Launching the MCP Service
6. Controlling the Robot via CLI
    - 6.1 CLI Command Overview
    - 6.2 Command Format
    - 6.3 Examples
7. Source Code Analysis
    - 7.1 MCP Service
    - 7.2 CLI Source Code

1. Course Content

Course Overview

- This section explains the use of the robot CLI (Command Line Interface) command tool. Through the command line interface, users can directly control various functions of the robot, such as chassis movement, robotic arm manipulation, visual recognition, navigation and positioning, and waste sorting.
- The CLI command tool encapsulates all MCP Tool interfaces, allowing for quick robot control without relying on AI models, facilitating functional testing and debugging.
- Compared to indirectly controlling robots through AI platforms such as OpenClaw, the CLI command tool provides a more direct and efficient interaction method, suitable for developers to perform functional verification, interface testing, and secondary development.

Learning Objectives

1. Understand the functional positioning and application scenarios of robot CLI command tools.
2. Be familiar with the functions, parameters, and usage formats of CLI commands.

2. Preparation

- Start the MicroROS chassis agent (if already started, no need to start again)

```
sh start_agent.sh
```

- Start nodes such as odometry, tf, robotic arm assistance, and camera nodes.

```
ros2 launch m3pro_bringup car_base.launch.py
```

3. Introduction to CLI Tools

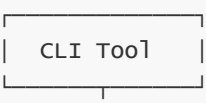
3.1 What is a Robot CLI Control Tool?

A robot CLI (Command Line Interface) control tool is a command-line-based robot control interface that uses the `robot_control` command to call various robot control tools registered with the MCP server. It encapsulates the robot's low-level control interface, transforming complex ROS2 communication into concise command-line instructions. Users can control the robot to perform various actions simply by entering commands in the terminal.

3.2 Why are CLI command tools needed?

- **Direct Remote SSH Control:** By remotely logging into the robot's terminal via SSH, users can directly execute CLI commands to control the robot.
- **AI-Independent Direct Control:** Bypassing AI platforms such as OpenClaw, users can rapidly manipulate the robot directly through the command-line interface, thereby reducing system dependencies and latency.
- **Independent Function Testing:** Specific functions can be invoked individually for testing purposes; this avoids the "black box" nature of AI-driven operations, making it easier to pinpoint and troubleshoot issues.
- **Rapid Interface Verification:** The CLI allows for the rapid iteration and testing of all robot control interfaces—verifying both functional completeness and parameter ranges—to ensure that every module is operating correctly.
- **Native AI-to-CLI Compatibility:** AI models are inherently compatible with command-line interfaces; this enables AI to directly generate and execute CLI commands to control the robot, facilitating highly efficient collaboration between AI and the CLI.

4. Communication Architecture for CLI-Controlled Robots

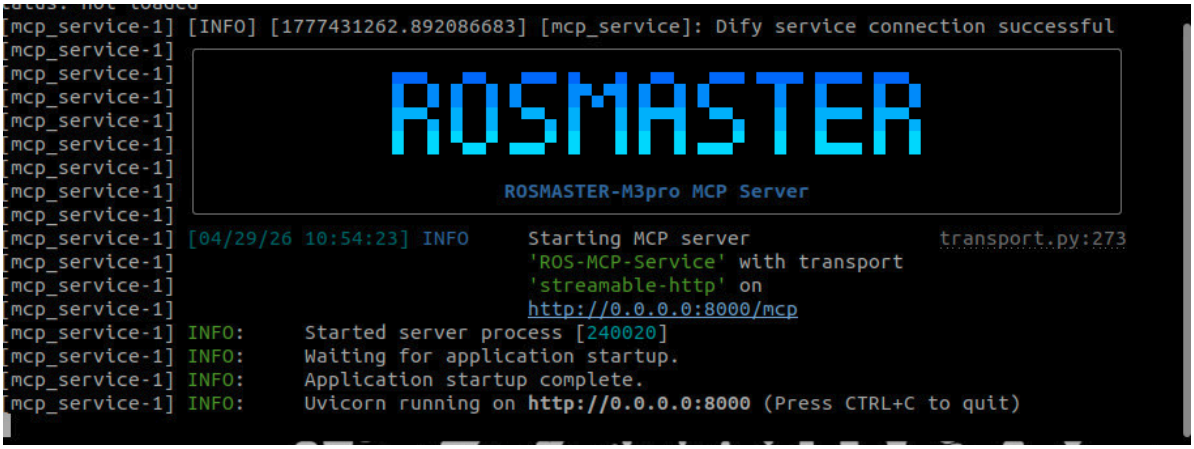


```
        | MCP Protocol
        ▼
┌──────────────┐
|  MCP Server  |
└──────┬───────┘
       |
       ▼
┌──────────────────────────┐
| ROS2 Communication Stack |
| Topics/Services/Actions  |
└──────┬───────────────────┘
       |
       ▼
┌──────────────────────┐
| Hardware Driver Layer |
└──────┬───────────────┘
       |
       ▼
┌─────────────────┐
| Robot Hardware |
└─────────────────┘
```

5. Launching the MCP Service

- Launch via Terminal

```
ros2 launch m3pro_bringup mcp_service.launch.py
```



6. Controlling the Robot via CLI

6.1 CLI Command Overview

- This section provides a summarized overview of the CLI commands. For complete documentation on CLI usage, please refer to the "CLI Command Summary" document located in the tutorial folder for this section.

Command	Parameters	Function
MoveWithSpeed	`--linear-x`, `--linear-y`, `--angular-z`, `--duration`	Control the robot's omnidirectional base movement via velocity commands
Move	`--distance`, `--direction`	Control the robot's base to move a specified distance
GetCurrentArmJointAngles	None	Retrieve the current angles of each robotic arm joint
GetEndEffectorPose	None	Retrieve the pose (position and orientation) of the end-effector (including gripper and camera)
ControlSixArmJoint	`--arm-joint-1`~`--arm-joint-6`, `--runtime`	Simultaneously control all 6 robotic arm joints; Joint 6 controls the gripper angle
ControlSingleArmJoint	`--arm-joint-id`, `--angle`, `--runtime`	Set a single robotic arm joint to a specified angle
Place	`--place-x`, `--place-y`, `--place-z`	Place an object at a specified coordinate position
Pick	`--x1`, `--y1`, `--x2`, `--y2`	Pick up an object
AdjustCameraView	`--pitch`, `--yaw`	Adjust the camera's field of view via pitch and yaw angles
SeeWhat	None	Capture an image from the camera and return the image's save path
InitArmPose	None	Restore the robotic arm to its preset initial pose
GetBbox	`--query`	Detect and return the bounding box coordinates of a target object based on a description
GetPlacePoint	`--query`	Retrieve placement coordinates and check if they lie within the robotic arm's reachable workspace
AdjustChassisFitArmRange	`--x`, `--y`, `--z`	Quickly adjust the mobile chassis to bring the target point within the robotic arm's working range
RecordMapLocation	`--name`, `--symbol`	Save the current position to the map mapping file

Command	Parameters	Function
GetMapMapping	None	Retrieve the mapping relationship between all location names and their corresponding identifiers
Navigation	`--location`	Navigate to a target location using its identifier
GetWasteRecognitionResults	None	Retrieve waste recognition results from the YOLO model
GraspWaste	`--waste-name`	Grasp a waste object
PlaceWaste	None	Place a waste object
TTS	`--text`	Robot voice broadcast
Rotate	`--angle`	Rotate in place by a specified angle
TargetTrack	`--x1-or-cmd`, `--y1`, `--x2`, `--y2`	Visual tracking of a target object
GetTargetDist	`--query`	Calculate the distance between a target object and the robot chassis based on the object's description
GraspArUcoTarget	`--target-id`	Grasp an AprilTag target with a specified ID
GetAprilTagIDs	None	Retrieve the IDs of AprilTag tags currently within the field of view
FollowLine	`--color`	Follow a colored line on the ground

6.2 Command Format

All CLI commands adhere to a unified command format:

```
robot_control call-tool <command_name> --param1 <value1> --param2 <value2> ...
```

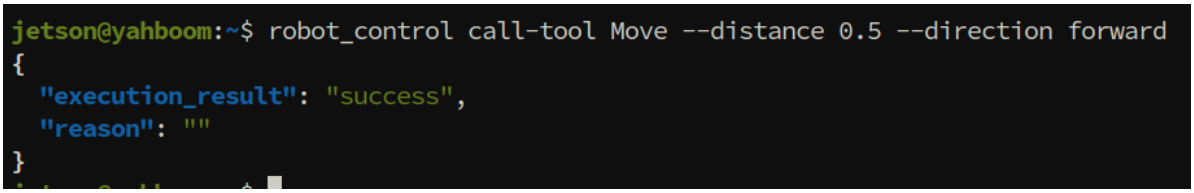
- `robot_control`: The main command for the CLI tool.
- `call-tool`: Indicates the subcommand for invoking a specific tool.
- `<command_name>`: Corresponds to the specific tool name registered on the MCP server side (e.g., `Move`, `SeeWhat`, `TTS`, etc.).
- `--param <value>`: Key-value pairs for parameters passed as needed; the number and types of parameters vary for different commands.

> Use `robot_control list-tools` to view a list of all available CLI commands.

6.3 Examples

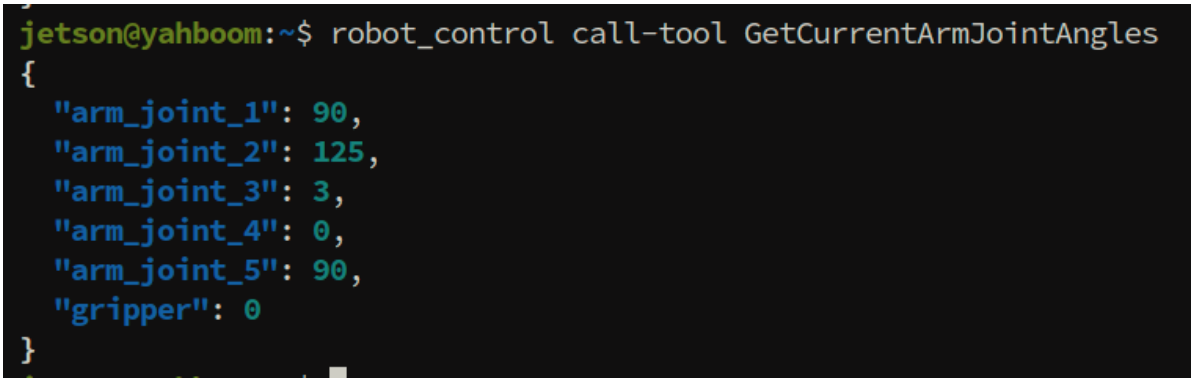
- Move the robot chassis forward by 0.5 meters:

```
robot_control call-tool Move --distance 0.5 --direction forward
```



- Retrieve the angles of all joints on the robotic arm:

```
robot_control call-tool GetCurrentArmJointAngles
```



> [!TIP]
>
> - When accessing the MCP server via the CLI, the CLI tool requires importing Python libraries during initialization, resulting in a "cold start" delay of approximately 2–3 seconds.

7. Source Code Analysis

7.1 MCP Service

- For details, please refer to [03-openclaw: Integrating Robot MCP Interfaces], where this topic has been explained in depth.

7.2 CLI Source Code

The source code for the CLI tool registers various robot control functions as "MCP Tools" via the `register_actions()` function located in `mcp_action.py`. Here, we use the `Move` tool (which moves the robot chassis a specified distance) as an example to explain the source code:

```
@mcp.tool(description='''Control the robot chassis to move a specified
distance''', timeout=30.0)
def Move(
    distance: float=Field(description="Moving distance in meters"),
    direction: str=Field(default="forward",
description="forward/backward/left/right")
) -> common_response:
    action_controller.move_distance(distance, direction)
    return common_response(execution_result="success", reason="")
```

Code Explanation:

1. **@mcp.tool()**: A decorator from the FastMCP framework used to register this function as a "Tool." The `description` parameter defines a functional description of the tool for the AI model to interpret, while `timeout` sets the maximum duration allowed for the operation.
2. **Function Parameters**: `distance` specifies the moving distance (in meters), and `direction` specifies the direction of movement (forward/backward/left/right). Both parameters utilize `Field` to define their default values and descriptions.
3. **action_controller**: An instance of the ROS2 controller created via `ROSContext`; it encapsulates the Action-based communication interface with the robot's hardware.
4. **common_response**: A standardized return format. `execution_result` indicates the outcome of the execution (success/failure), while `reason` is used to provide details in the event of a failure.